

2. ELEMENTELE LIMBAJULUI DE ASAMBLARE

-- suport de laborator --

Registrele generale

Procesorul Intel **8086** are o serie de registre generale pe 16-biti fiecare, utilizati în diferite scopuri. Aceste registre sunt zone de memorie de dimensiune fixa, aflate in procesor si care pot fi accesate din aplicatiile scrise in limbaj de asamblare. In tabelul următor ele sunt grupate în:

- 4 registre de segment: CS, DS, SS, ES
- 3 registre de index: IP, SP, BP
- 4 registre de bază: AX, BX, CX, DX
- 2 registre (de) index: SI, DI

Registre de Segment		
CS	Code Segment	Numar pe 16-biti ce reprezinta adresa segmentului de cod activ.
DS	Data Segment	Numar pe 16-biti ce reprezinta adresa segmentului de date activ.
SS	Stack Segment	Numar pe 16-biti ce reprezinta adresa segmentului de stiva activ.
ES	Extra Segment	Numar pe 16-biti ce reprezinta adresa segmentului suplimentar activ.
Registre index		
IP	Instruction Pointer	Numar pe 16-biti ce reprezinta offset-ul instructiunii urmatoare de executat în cadrul segmentului de cod.
SP	Stack Pointer	Numar pe 16-biti ce reprezinta offset-ul din cadrul stivei.
BP	Base Pointer	Utilizat pentru a transfera în/din stiva date.
Registre de baza		
AX	Accumulator Register	Calcule si operatii de intrare/iesire
BX	Base Register	Utilizat ca index
CX	Count Register	Utilizat in instructiunile de ciclare
DX	Data Register	Operatii de intrare/iesire, înmultire/împartire.
Registre index		
SI	Source Index	Utilizat in operatiile cu siruri pentru a parcurge sursa.
DI	Destination Index	Utilizat in operatiile cu siruri pentru a parcurge sursa.

Registrele de baza pot fi accesate si pe octeti, adică octetul inferior (L, Low) format din cei 8 biti mai puțin semnificativi, respectiv octetul superior (H, High) al celor 8 biti mai semnificativi.

AX	
AH	AL
BX	
BH	BL
CX	
CH	CL
DX	
DH	DL

De exemplu, dacă registrul **AX** conține valoarea **1234h** atunci **AH** are valoarea **12h** și **AL** are **34h**.

Un registru pe 16 biti numit **registru FLAG** este un registru de stare utilizat pentru a urmări (a controla) execuția instrucțiunilor. Bitii acestui registru se numesc *flag-uri* sau *biti de stare* sau, încă, *indicatori de stare* și pot avea valoarea 1 (**SET**) sau 0 (**NOT SET**) după cum indică starea curentă a execuției programului curent.

Registru de FLAG-uri		
Abr.	Nume	Rang bit
OF	Overflow Flag	11
DF	Direction Flag	10
IF	Interrupt Flag	9
TF	Trap Flag	8
SF	Sign Flag	7
ZF	Zero Flag	6
AF	Auxiliary Carry	4
PF	Parity Flag	2
CF	Carry Flag	0

Semnificatia indicatorilor de stare este urmatoarea:

- **CF** (Carry Flag, indicator de transport) semnifica un transport sau un împrumut în, respectiv din bitul cel mai semnificativ al rezultatului.
- **PF** (Parity Flag, indicator de paritate) este pozitionat după cum bitii din cel mai puțin semnificativ octet al rezultatului sunt în număr par (pozitionare pe 1) sau impar (pozitionare pe 0).
- **AF** (Adjust Flag, indicator de ajustare) este folosit în aritmetica zecimală și semnifica un transport sau un împrumut în, respectiv din bitul 4 (cel mai semnificativ bit al jumătății celei mai puțin semnificative) al rezultatului.
- **ZF** (Zero Flag, indicator de zero) indică dacă rezultatul unei operații a fost sau nu zero.
- **SF** (Sign Flag, indicator de semn) are aceeași valoare ca bitul cel mai semnificativ al rezultatului (bitul de semn): 0 – pozitiv, 1 – negativ.

- **TF** (Trap Flag, indicator de urmarire a executiei) este folosit la depanarea programelor prin executia lor pas cu pas – daca este setat, procesorul forteaza automat o exceptie dupa executia fiecărei instructiuni.
- **IF** (Interrupt Flag, indicator de întreruperi) precizeaza daca procesorul ia în considerare sau nu întreruperile externe.
- **DF** (Direction Flag, indicator de directie) precizeaza sensul (0 – crescator sau 1 –descrescator) în care este modificat contorul de adrese la operatiile cu siruri.
- **OF** (Overflow Flag, indicator de depasire) semnifica depasirea domeniului admisibil la reprezentarea rezultatului unei operatii aritmetice cu sau fara semn. Practic, este pozitionat pe 1 daca apare un transport înspre bitul cel mai semnificativ al rezultatului din bitul vecin, dar nu si din bitul cel mai semnificativ spre CF sau invers, dinspre bitul cel mai semnificativ spre CF, dar nu si spre bitul cel mai semnificativ din bitul vecin. Similar, la împrumut, este pozitionat pe 1 daca apare un transport de la bitul cel mai semnificativ la bitul vecin, dar nu si înspre bitul cel mai semnificativ dinspre CF sau invers, dinspre CF spre b.c.m.s., dar nu si dinspre bitul cel mai semnificativ spre bitul vecin.

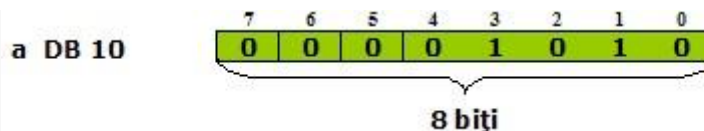
Tipuri de date in limbaj de asamblare

1. Bit
2. Nibble – grup de 4 biti; utilizat în codificarea numerelor în format BCD (Binary Coded Decima)
3. Byte (octet) – grup de 8 biti; definire cu **db**;
4. Word (cuvant) – grup de 16 biti sau 2 octeti; definire cu **dw**;
5. Double (dublu cuvant) – grup de 32 biti sau 4 octeti; definire cu **dd**;
6. Quad – grup de 64 biti sau 8 octeti; definire cu **dq**.

O descriere detaliata a tipurilor de date fundamentale ale limbajului de asamblare:

Tipul de data – BYTE

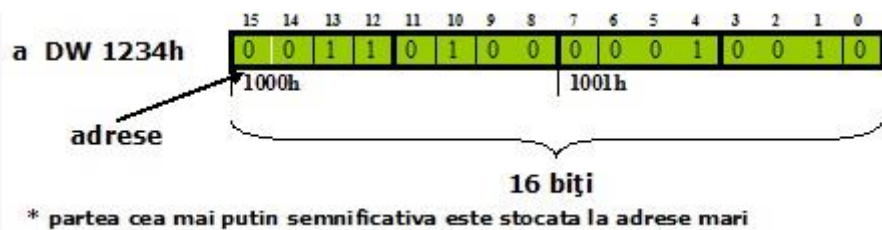
- **Spatiu ocupat:** 1 octet
- **Definire:** a DB 10
- **Interpretare:**
 - Intreg pe 8 biti cu sau fara semn
 - Caracter ASCII



Tipul de date Byte – limbaj de asamblare

Tipul de data – WORD

- **Spatiu ocupat:** 2 octeti
- **Definire:** a DW 1234h
- **Interpretare:**
 - Intreg pe 16 biti cu sau fara semn
 - Secventa de 2 caractere ASCII
 - Adresa de memorie pe 16 biti



Tipul de date Word – limbaj de asamblare

Tipul de data – DOUBLE-WORD

- **Spatiu ocupat:** 4 octeți
- **Definire:** a DD 12345678h
- **Interpretare:**
 - Intreg pe 32 biți cu sau fără semn
 - Număr real simplă precizie
 - Adresa de memorie pe 32 biți



Tipul de date Double – limbaj de asamblare

Tipul de data QUAD-WORD

- **Spatiu ocupat:** 8 octeți
- **Definire:** a DQ 12345678
- **Interpretare:**
 - Intreg pe 64 biți cu sau fără semn
 - Număr real dublă precizie



Tipul de date Quad – limbaj de asamblare

Exemplu 1.

Program in limbaj de asamblare ce calculeaza valoarea expresiei $e = a + b$ in care variabilele au valori fixe (sunt operanzi transmisi prin valoare):

```
; programul determina valoarea expresiei e, unde e=a+b
;-----
.model small
.286
.stack 100h
.data
    a db 10
    b db 35

    e db ?

.code
    mov AX,@data
    mov DS,AX

    mov AL,a
    add AL,b
    mov e,AL

    mov AX,4c00h
    int 21h

end
```

Explicarea programului:

.model small: Liniile care incep cu “.” reprezinta instructiuni speciale care indica programului assembler anumite informatii, descrise de cuvintele cheie ce urmeaza, cu privire la programul de construit. in aceasta situatie **model** indica faptul ca urmeaza a se indica modelul de memorie (si cantitatea de memorie) utilizat de program. Acest program necesita un spatiu redus de memorie, fapt indicat prin **small**. Un model de memorie specifica modul in care codul si datele sunt adresate, ori sunt in acelasi segment fizic ori in mai multe segmente. Când toate datele (sau tot codul) se afla in acelasi segment atunci elementele sunt adresate prin adrese *near* date de deplasarea lor (offset) fata de adresa de inceput a segmentului. Daca se utilizeaza mai multe segmente atunci elementele sunt adresate prin adrese *far* date de adresa segmentului si offset. Tipuri standard de modele sunt: SMALL, MEDIUM, COMPACT si LARGE.

	Model de memorie			
Tip adrese	SMALL	MEDIUM	COMPACT	LARGE
Adrese COD	near	near	far	far
Adrese DATE	near	far	near	far

.stack: Alta linie care descrie programul. Instructiunea indica locul in care incepe segmentul de stiva. Acesta este utilizat ca zona temporara de stocare a rezultatelor intermediare sau pentru a conserva starea sistemului (descrisa de valorile din registre) inainte de a efectua operatii care sa altereze datele existente. De asemenea stiva este utilizata si in transferul parametrilor catre si din proceduri. Indiferent daca este utilizata sau nu este obligatorie definirea segmentului de stiva deoarece un program .EXE trebuie sa aiba o stiva. Optional se poate defini si dimensiunea stivei ca numar de octeti. Daca nu se specifica dimensiunea se iau implicit 1024 de bytes.

.data : indica faptul ca incepe segmentul de date. Implicit, reprezinta terminarea segmentului de stiva. In segmentul de date sunt definite variabilele cu care se lucreaza in program.

a db 10: se defineste o variabila de tip byte cu valoarea zecimala 10.

.code : indica inceperea segmentului de cod si implicit, terminarea segmentului de date. Acest segment contine instructiunile programului.

mov AX, @data : Instructiunea incarca in registrul AX adresa segmentului de date. Alte simboluri predefinite @code – adresa segmentului de cod.

mov DS,AX: Instructiunea incarca in registrul segment DS adresa segmentului de date din registrul AX. Operatia este necesara deoarece nu este permisa incarcarea registrului DS cu o valoare constanta (adresa).

mov AL,a: se pune in registrul AL valoarea lui a.

add AL,b: se aduna la valoarea din registrul AL, valoarea lui b.

mov e,AL: variabila e este initializata cu valoarea din registrul AL

mov AX, 4c00h : incarca in registrul AX valoarea hexazecimala 4c00h. Acest lucru este necesar apelarii ulterioare a rutinei 21h.

int 21h : Apelul intreruperii 21h. In registrul AH se gaseste valoarea 4ch deoarece AX are valoarea 4c00h. Pentru rutina DOS acest lucru semnifica iesire din program. Valoarea din AL, adica 00h, reprezinta codul de iesire din program ce indica terminare executie fara eroare.

.end : marcheaza sfârșitul fisierului sursa. **ATENȚIE**: Daca se specifica numele unei etichete definita anterior (de obicei inainte de prima instructiune) atunci aceasta reprezinta adresa din CS la care porneste executia programului. De exemplu programul

```
...  
.code  
    mov AX,@data  
    mov DS,AX  
Start:  
    mov AL,a  
    add AL,b  
    mov e,AL  
...  
end Start
```

ruleaza începând cu instructiunea *mov AL,a*.

Exemplu 2

Afișează un mesaj la consolă.

```
; programul afiseaza un mesaj  
;-----  
.model small  
.stack  
.data  
mesaj    db "Afisare mesaj !!!", "$"  
  
.code  
  
main     proc  
    mov  AX, seg mesaj  
    mov  DS, AX  
  
    mov  AH, 09  
    lea  DX, mesaj  
    int  21h  
  
    mov  AX, 4c00h  
    int  21h  
main     endp  
end main
```

Explicarea programului:

main proc : codul poate fi structurat prin intermediul procedurilor. Pentru a face analogie cu programul C care trebuie sa contina obligatoriu o procedura numita *main*, se defineste procedura si in programul assembler. Instructiunea indica inceputul subprogramului numit *main*. Sfarsitul subprogramului este indicat de instructiunea *end main*.

mesaj db "

Afisare mesaj !!!": defineste o variabila sir de caractere numita *message* ce contine textul **Afisare mesaj !!!**.

mov AX, seg mesaj : Instructiunea incarca in registrul AX adresa segmentului de date. Reprezinta o alternativa la instructiunea *mov AX, @data* pentru ca instructiunea *seg* intoarce adresa segmentului in care se afla definita variabila respectiva. Situatia este posibila pentru ca fiecare variabila este identificata printr-o adresa de forma *segment:offset*. Fara a incarca registrul DS cu adresa segmentului activ de date nu se incarca corect adresa variabilei *mesaj*.

mov AH, 09 : incarca in registrul AH valoarea constanta 09. Este necesar pentru a afisa un mesaj pe ecran utilizând intreruperea DOS 21h.

lea DX, message : Instructiunea LEA (Load Effective Address) incarca in registrul DX offset-ul din cadrul segmentului de date la care se gaseste variabila *mesaj*. Acest lucru este necesar pentru a afisa mesajul pe ecran utilizând intreruperea 21h.

int 21h : Instructiunea genereaza o intrerupere DOS. Procesorul apeleaza rutina indicata de numarul intreruperii, in acest caz o rutina DOS. Procedura verifica registrul AH pentru a vedea ce trebuie sa faca. In acest caz, valoarea 09 din AH indica faptul ca trebuie sa scrie pe ecran un sir de biti.

Instructiuni utilizate in cele 2 exemple

Instructiunea MOV

- realizeaza principalele operatii de transfer a valorilor;
- este atât de utilizata încât este imposibil sa scrii un program assembler fara aceasta instructiune;
- forma analitica a instructiunii este:

MOV destinatie, sursa

si realizeaza copierea valorii de la *sursa* la *destinatie* fara a modifica sursa.

- permite multiple combinatii de tipuri ale sursei si destinatiei (registru, valoare imediata, locatie de memorie) insa exclude situatiile:
 - destinatia nu poate fi segmentul CS; cum segmentul CS contine intotdeauna adresa codului de executat nu este indicat a se modifica aceasta valoare in timpul prelucrării; singurul mod de a modifica CS este prin intermediul instructiunilor *int*, *jmp* sau *call*;
 - destinatia si sursa nu pot fi in acelasi timp operanzi din memorie (variabile definite in segmentul de date); aceasta restrictie se aplica tuturor instructiunilor procesoarelor 80x86 care necesita doi operanzi; exemplu:

```
.data
    vb1 DW 300
    vb2 DW ?
...
mov vb2, vb1      ;EROARE
```

pentru a evita situatia se utilizeaza un registru asemenea unei zone temporare; de exemplu:

```
.data
    vb1 DW 300
    vb2 DW ?
...
mov AX, vb1
mov vb2, AX
```

- daca sursa este o valoare imediata (constanta valorica), destinatia nu poate fi unul dintre registrele de segment (CS, DS, ES, SS); pentru a evita situatia se utilizeaza un registru;
- destinatia nu poate fi un operand imediat (constanta valorica); exemplu:

```
mov 5, AX ;EROARE
```

- cea mai rapida combinatie este MOV registru, registru;

Instructiunea XCHG

- interchimba valorile din *sursa* si *destinatie*;
- forma analitica:

XCHG destinatie, sursa

- nu afecteaza nici un Flag bit;

Instructiunile LDS si LES

- sunt printre putinele instructiuni care prelucreaza un dublu cuvânt (32 de biti); transfera dublu-cuvântul din memorie catre 2 registre de 16 biti; valoarea din cei doi octeti superiori ai dublu-cuvântului este copiată într-unul din registrele de segment (fie DS sau ES in functie de instructiune LDS sau LES); cuvântul (16 biti) mai puțin semnificativ este copiat într-un registru general indicat ca operand destinatie in instructiune; de obicei dublu-cuvântul este un pointer ce contine adresa unei variabile data de segment:offset (adresa segmentului pe 16 biti si offsetul in cadrul segmentului tot pe 16 biti);
- incarca adresa in segment;
- forma analitica:

LDS registru, sursa

LES registru, sursa

unde sursa este reprezentata de o variabila definita pe 32 de biti;

- registrul FLAG este neafectat;
- restrictie: *registru* nu poate fi unul din registrele de segment;
- exemplu:

```
mesaj db "Afisare mesaj !!!", "$"

pointer_mesaj dd mesaj

...

lds DX, pointer_mesaj ; incarca in DS adresa segmentului in care se afla mesaj si
in DX offset-ul
```

sau daca DS este deja incarcat cu adresa segmentului atunci

```
les DX, mesaj ;incarca in ES adresa segmentului si in DX offset-ul
```

ultima instructiune poate fi inlocuita cu:

```
mov DX, OFFSET mesaj ;OFFSET returneaza offset-ul variabilei mesaj
```

Instructiunea ADD

- aduna la operandul destinatie valoarea operandului sursa;
- adunarea se realizeaza pe 16 biti;
- forma analitica:

ADD destinatie, sursa

- permite multiple combinatii de tipuri ale sursei si destinatiei (registru, valoare imediata, locatie de memorie) insa exclude situatiile:

- o destinatia nu poate fi segmentul CS; cum segmentul CS contine intotdeauna adresa codului de executat nu este indicat a se modifica aceasta valoare in timpul prelucrarii.; singurul mod de a modifica CS este prin intermediul instructiunilor *int*, *jmp* sau *call*;
- o destinatia si sursa nu pot fi in acelasi timp operanzi din memorie (variabile definite in segmentul de date); aceasta restrictie se aplica tuturor instructiunilor procesoarelor 80x86 care necesita doi operanzi; exemplu:

```
.data
    vb1 DW 300
    vb2 DW ?
...
add vb2, vb1      ;EROARE
```

pentru a evita situatia se utilizeaza un registru asemenea unei zone temporare; de exemplu:

```
.data
    vb1 DW 300
    vb2 DW ?
...
mov AX, vb1
add vb2, AX
```

- daca sursa este o valoare imediata (constanta valorica), destinatia nu poate fi unul dintre registrele de segment (CS, DS, ES, SS); pentru a evita situatia se utilizeaza un registru;
- destinatia nu poate fi un operand imediat (constanta valorica); exemplu:

```
add 5, AX      ;EROARE
```

- registru FLAG modificat: OF, CF, PF, SF, AF, ZF;

Instructiunea SUB (SUBstract)

- scade din operandul destinatie valoarea operandului sursa;
- forma analitica:

SUB destinatie, sursa

- restrictii identice ca la instructiunea ADD;
- exemple:

```
.data
    vb1 DW 300
    vb2 DW 50
...
mov AX, vb1
mov CX, vb2
sub vb1, 50
sub AX, CX
sub AX, vb1
sub AX, 10
```

Instructiunea NEG

- utilizat pentru a nega operandul, scazând-ul din valoarea 0;
- forma analitica:

NEG operand

- registru FLAG afectat: OF, SF, ZF, AF, PF, CF;
- operandul trebuie sa fie registru sau variabila;
- când operandul contine valoarea minima posibila aceasta nu este afectata;

Instructiunea MUL (MULTiply)

- multiplica o valoare unsigned din AL (când operandul este de tip byte) sau din AX (când operandul este de tip word) cu valoarea operandului specificat; rezultatul este returnat in AX

când operandul este de tip byte si in DX:AX (cei 2 octeti superiori in DX) când acesta este de tip word;

- forma analitica:

MUL operand

- registru FLAG modificat: OF si CF sunt setati daca partea superioara a rezultatului (DX daca operandul este de tip word, sau AH este de tip byte) este diferita de 0;alte flag-uri SF, ZF, AF, PF;
- operandul poate fi alt registru sau o variabila;
- daca valoarea inmultita are semn atunci se utilizeaza IMUL;

Instructiunea DIV (DIVide)

- imparte o valoare unsigned din AX (când operandul este de tip byte; in acest caz DX trebuie sa contina valoarea 0) sau din DX:AX (când operandul este de tip word) cu valoarea operandului specificat; câtul este returnat in AL iar restul in AH când operandul este de tip byte si in DX:AX (DX contine restul si AX câtul) când acesta este de tip word;
- forma analitica:

DIV operand

- registru FLAG modificat: nedefiniti;
- operandul poate fi alt registru sau o variabila;
- in cazul operandului de tip word memorarea deimpartitului in DX:AX se face cu instructiunea **cwd** (convert word to double) ce extinde valoarea din AX in zona DX:AX; exemplu:

```
vb1 dw 55
vb2 dw 15
cat dw ?
rest dw ?
...
mov AX, vb1
cwd
div vb2
mov cat, AX
mov rest, DX
```

Instructiunea ROR (ROtate Right)

- utilizata pentru a roti la dreapta bitii destinatiei de atâtea ori cât este specificat; cum fiecare bit este rotit la dreapta, cel mai puțin semnificativ bit al destinatiei este copiat in locul celui mai semnificativ bit si in CF (carry flag);
- forma analitica:

ROR destinatie, contor

- destinatia poate fi registru sau variabila; contorul poate fi constanta sau registrul CL; exemple:

```
vb dw 1234h
...
mov CL, 4
ror AX, CL
ror AX, 4
ror vb, CL
ror vb, 4
```

- registru FLAG: OF este setat doar daca contorul are valoarea 1 si bitul cel mai semnificativ al destinatiei este diferit de bitul vecin; pentru alte valori nu exista regula de modificare a lui OF; se modifica si CF;

Instructiunea ROL (ROtate Left)

- utilizata pentru a roti la stânga bitii destinatiei de atâtea ori cât este specificat; cum fiecare bit este rotit la stânga, cel mai semnificativ bit al destinatiei este copiat in locul celui mai puțin semnificativ bit si in CF (carry flag);
- analogie cu ROR;

Instructiunea SAL sau SHL (Shift Arithmetic Left si SHift Left)

- se utilizeaza pentru a muta la stânga bitii din destinatie cu atâtea pozitii câte sunt specificate in contor; cu fiecare pozitie se adauga la stânga in bitul cel mai puțin semnificativ valoarea 0;
- forma analitica:
$$SAL \text{ destinatie, contor}$$
- registrul FLAG: SF, ZF si PF; cel mai semnificativ bit al destinatiei este mutat in CF; OF este setat daca contorul are valoarea 1 iar bitul de semn isi pastreaza valoarea;
- utilizat pentru a inmulti destinatia o putere a lui 2;
- destinatia poate fi registru sau variabila; contorul poate fi constanta sau registrul CL;

Instructiunea SAR (Shift Arithmetic Right)

- se utilizeaza pentru a muta la dreapta bitii din destinatie cu atâtea pozitii câte sunt specificate in contor; cu fiecare pozitie se adauga la dreapta in vecinul bitului cel mai semnificativ valoarea 0; bitul cel semnificativ (bitul de semn) isi pastreaza valoarea;
- forma analitica:
$$SAR \text{ destinatie, contor}$$
- registrul FLAG: SF, ZF si PF; cel mai puțin semnificativ bit al destinatiei este mutat in CF; OF este setat daca contorul are valoarea 1 iar bitul de semn si vecinul sau sunt diferiti;
- utilizat pentru a imparti un numar negativ (destinatia) la o putere a lui 2;
- destinatia poate fi registru sau variabila; contorul poate fi constanta sau registrul CL;

Instructiunea SHR (Shift Righth)

- se utilizeaza pentru a muta la dreapta bitii din destinatie cu atâtea pozitii câte sunt specificate in contor; cu fiecare pozitie se adauga in vecinul bitului cel mai semnificativ valoarea 0;
- forma analitica:
$$SHR \text{ destinatie, contor}$$
- registrul FLAG: SF, ZF si PF; cel mai puțin semnificativ bit al destinatiei este mutat in CF; OF este setat daca contorul are valoarea 1 iar bitul de semn si vecinul sau sunt diferiti;
- utilizat pentru a imparti un numar pozitiv (destinatia) la o putere a lui 2;
- destinatia poate fi registru sau variabila; contorul poate fi constanta sau registrul CL;

Întreruperi

CLASIFICAREA INTRERUPERILOR

- intreruperi hardware - generate in mod automat ca raspuns la aparitia unor cauze externe microprocesorului;
- exceptii - generate in mod automat ca raspuns la aparitia unor evenimente de tip intern;
- intreruperi software - presupune un transfer de control al executiei, inițiat de programator catre o rutina speciala;

Întreruperi hardware

int 08h	întreruperea de ceas pentru a gestiona bine ora sistem
int 09h	întreruperea de la tastatură generată la fiecare apăsare și eliberare a unei taste
int 0Bh int 0Ch	deservesc porturile seriale comunicarea se face bit după bit (mouse, USB)
int 0Dh int 0Fh	deservesc porturile paralele transmisia simultană a mai multor biți (imprimanta)
int 0Eh	întreruperea de dischetă
int 76h	întreruperea de harddisk generată la încheierea operațiilor de citire scriere

Excepții

int 00h	întreruperea generată când apare o condiție de împărțire la 0 Poate să apară în 3 situații: - încercare de împărțire la 0; - depășirea câtului la împărțire; - emiterea explicită a acestei întreruperi.
int 01h	este inițiată de procesor după fiecare instrucțiune mașină dacă flag-ul TF=1
int 02h	este generată de fiecare dată când apare o condiție nemascabilă Mascabil=pot să inhib apariția întreruperii
int 03h	este folosită de către depanatoare pentru stabilirea de puncte de întrerupere a execuției
int 04h	se emite când are loc o depășire în cadrul unor operații aritmetice
int 06h	apare când am un cod de instrucțiune ilegal

Intreruperi DOS

■ întreruperi DOS

- int 21h - principala întrerupere DOS;
- int 20h - are ca și efect terminarea execuției unui program;
- int 25h - citire de pe disk, scriere pe disk, lucrul cu
mouse-ul, terminarea execuției programului, lăsându-l
rezident;

ÎNTRERUPEREA 21H

- funcții de gestiune a memoriei (alocare eliberare);
- funcții de gestiune a procesului (terminarea unui program
4ch);
- funcții specifice pentru directoare și fișiere;
- informații despre tabela de alocare a fișierelor;
- intrări și ieșiri cu periferice de tip caracter;
- informații despre sistem - returnează versiunea sistemului
de operare;